

OSSP-PRACTICAL-WEEK2

Task-1: Develop a C program that uses the system calls `open()`, `read()`, `write()`, and `close()` to copy the contents of one file to another. Explain how control transitions between user space and kernel space during execution. Source code:

```
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>

int main() {
    int source, destination;
    char buffer[1024];
    ssize_t bytesRead, bytesWritten;

    source = open("source.txt", O_RDONLY);

    if (source < 0) {
        perror("Error opening source file");
        return 1;
    }

    destination = open("destination.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (destination < 0) {
        perror("Error opening destination file");
        close(source);
        return 1;
    }

    printf("Source file opened successfully.\n");
    printf("Destination file opened successfully.\n");

    while ((bytesRead = read(source, buffer, sizeof(buffer))) > 0) {
        bytesWritten = write(destination, buffer, bytesRead);

        if (bytesWritten != bytesRead) {
            perror("Error writing to destination file");
            close(source);
            close(destination);
            return 1;
        }
    }

    if (bytesRead < 0) {
        perror("Error reading source file");
    } else {
        printf("File contents copied successfully.\n");
    }

    close(source);
    close(destination);

    printf("Files closed successfully.\n");

    return 0;
}
```

Output:

```
anuan@Anushka:~$ nano file_copy.c
anuan@Anushka:~$
anuan@Anushka:~$ gcc file_copy.c -o file_copy
anuan@Anushka:~$ echo "Hello, this is a system call demonstration." > source.txt
anuan@Anushka:~$ ./file_copy source.txt destination.txt
File copied successfully.
anuan@Anushka:~$ cat destination.txt
Hello, this is a system call demonstration.
anuan@Anushka:~$
```

Observation: I learned how Linux system calls such as `open()`, `read()`, `write()`, and `close()` allow a C program to interact with files. I also understood that these

operations involve communication between user space and the Linux kernel, where the kernel provides the required file-handling services.

Task2: Use the strace utility to trace all system calls generated by the command cat sample.txt. Analyze the sequence of system calls and identify the kernel services involved.

Output:

```
anuan@Anushka:~$ echo "Hello from Linux system calls." > sample.txt
anuan@Anushka:~$ cat sample.txt
Hello from Linux system calls.
anuan@Anushka:~$ Hello from Linux system calls.
Command 'Hello' not found, did you mean:
  command 'hello' from snap hello (2.10)
  command 'hello' from deb hello (2.10-5build1)
  command 'jello' from deb jello (1.6.1-1)
See 'snap info <snapname>' for additional versions.
```

Observation: I learned how the strace utility can be used to observe the system calls made by a Linux command. I understood how commands such as cat interact with the Linux kernel through system calls to open, read, display, and close a file.